

Java Backend Architecture

Interview Series

Chapter 14.6

API Gateway & Service Discovery

50+ Interview Q&A

Code Examples

Architecture Diagrams

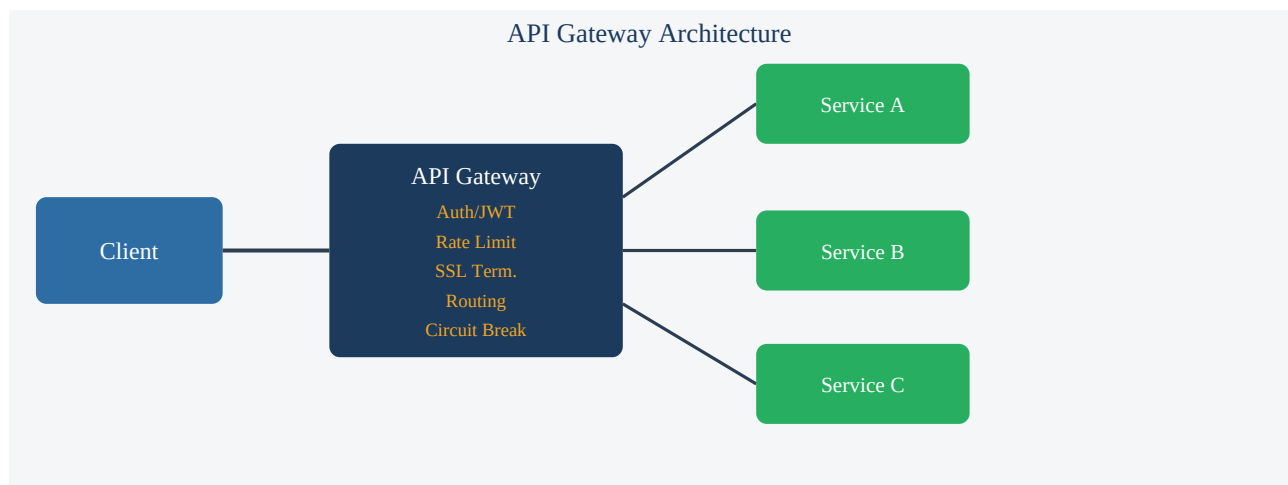
Advanced Topics

Chapter 14.6 - API Gateway & Service Discovery

Overview

An API Gateway acts as a single entry point for all client requests, centralising cross-cutting concerns such as authentication, rate limiting, SSL termination, request routing, and load balancing. Service discovery enables microservices to locate each other dynamically without hard-coded addresses. Circuit breakers prevent cascading failures by short-circuiting calls to unhealthy services.

API Gateway Architecture



Core API Gateway Responsibilities

- Authentication & Authorization: validates JWT tokens, OAuth2 scopes before forwarding
- Rate Limiting: token bucket or sliding window per client/IP; returns HTTP 429 on breach
- Request Routing: path/host-based routing to upstream microservices
- SSL Termination: decrypts HTTPS at the gateway; internal traffic may use plain HTTP
- Request/Response Transformation: header injection, payload mapping, protocol translation
- Load Balancing: round-robin, weighted, least-connections across service instances
- Circuit Breaking: stops forwarding to a failing upstream; returns cached/fallback response

Popular API Gateway Solutions

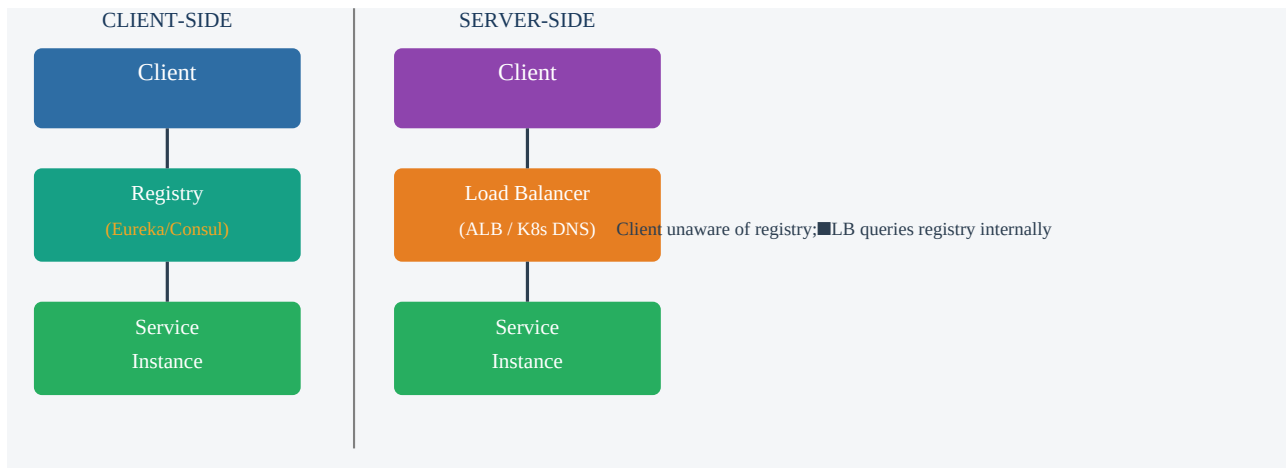
Gateway	Type	Key Feature	Use Case
Kong	Open-source, plugin-based	Lua plugins, declarative config	On-prem / K8s
AWS API Gateway	Managed SaaS	Lambda integration, IAM auth	AWS-native workloads
Nginx + Lua	DIY	Low latency, scripting	Custom routing logic
Spring Cloud Gateway	Java framework	Reactive, filter chain	Spring ecosystem
Apigee	Enterprise SaaS	Analytics, monetisation	Enterprise API mgmt

Spring Cloud Gateway - Filter Example

```
spring: cloud: gateway: routes: - id: order-service uri: lb://ORDER-SERVICE predicates: -  
Path=/api/orders/** filters: - StripPrefix=1 - name: RequestRateLimiter args:  
redis-rate-limiter.replenishRate: 10 redis-rate-limiter.burstCapacity: 20
```

```
@Component public class JwtAuthFilter extends AbstractGatewayFilterFactory<Object> { @Override  
public GatewayFilter apply(Object config) { return (exchange, chain) -> { String token =  
exchange.getRequest().getHeaders().getFirst("Authorization"); if  
(!jwtService.validate(token)) { exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);  
return exchange.getResponse().setComplete(); } return chain.filter(exchange); }; } }
```

Service Discovery Patterns



Aspect	Client-Side	Server-Side
Who queries registry?	Client/Service	Load Balancer / DNS
Load balancing	Client-side (Ribbon/SCL)	LB/Proxy (ALB, K8s)
Examples	Eureka + Ribbon, Consul	AWS ALB, Kubernetes Service
Coupling	Client knows registry	Client unaware of registry
Complexity	More client logic	Simpler client, heavier infra

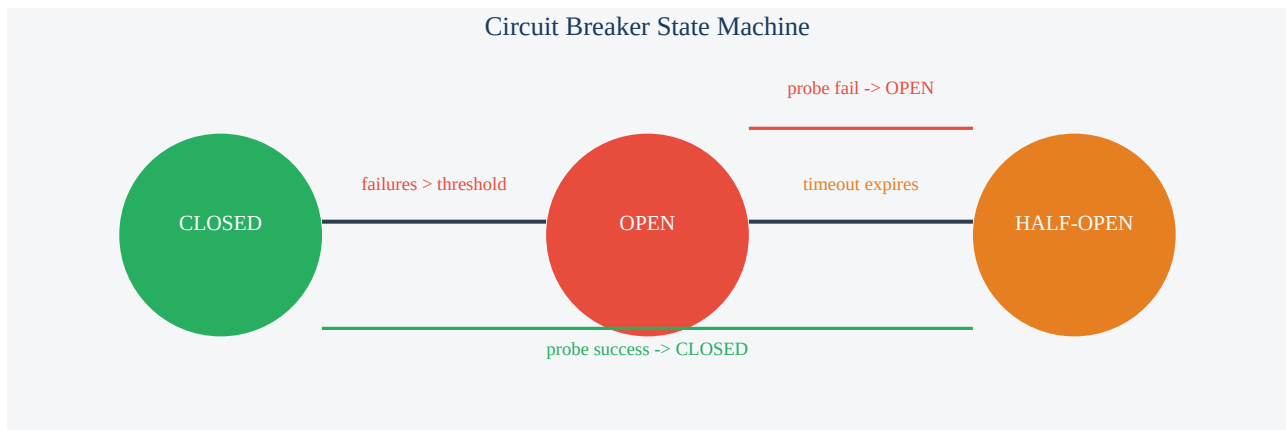
Eureka vs Consul

Eureka	Netflix OSS; AP (availability over consistency); self-preservation mode keeps stale data during partitions; REST API; Java-native.
Consul	HashiCorp; CP (consistency preferred); health checks built-in; KV store for configuration; multi-DC support; DNS + HTTP API.
Self-preservation	Eureka stops evicting instances if heartbeat loss exceeds threshold - avoids mass de-registration during network partition.
Health Check	Consul performs active health checks (HTTP/TCP/script); Eureka relies on heartbeat TTL only.

Spring Cloud Eureka Registration

```
// build.gradle implementation
"org.springframework.cloud:spring-cloud-starter-netflix-eureka-client" // application.yml
eureka: client: serviceUrl: defaultZone: http://eureka-server:8761/eureka/ instance:
preferIpAddress: true leaseRenewalIntervalInSeconds: 10 // Feign client (client-side LB)
@FeignClient(name = "INVENTORY-SERVICE") public interface InventoryClient {
@GetMapping("/api/inventory/{sku}") InventoryDto getStock(@PathVariable String sku); }
```

Circuit Breaker - Resilience4j



Circuit Breaker States

CLOSED	Normal operation. Requests pass through. Failure count is tracked.
OPEN	Failure rate exceeded threshold. All requests fail fast with fallback. No upstream calls.
HALF-OPEN	After timeout, a limited number of probe requests are allowed through to test recovery.
Threshold	Configurable: e.g., 50% failure rate over 10 calls triggers OPEN state.
Fallback	OPEN state returns cached result, default value, or error response immediately.

Resilience4j Circuit Breaker Code

```
# application.yml resilience4j: circuitbreaker: instances: inventoryService:
  slidingWindowSize: 10 failureRateThreshold: 50 waitDurationInOpenState: 30s
  permittedNumberOfCallsInHalfOpenState: 3
```

```
@Service public class OrderService { @CircuitBreaker(name = "inventoryService", fallbackMethod = "fallback") public InventoryDto checkInventory(String sku) { return inventoryClient.getStock(sku); } public InventoryDto fallback(String sku, Exception e) { log.warn("Circuit open for sku={}, returning default", sku); return new InventoryDto(sku, 0); } }
```

Rate Limiting Algorithms

Algorithm	Mechanism	Pros	Cons
Token Bucket	Tokens added at rate r ; request allowed if up to bucket size	Smooth burst	Burst can overwhelm
Leaky Bucket	Queue processed at fixed rate	Smooth output rate	Queue latency, drops
Fixed Window	Counter reset every window	Simple	Boundary spike (2x)
Sliding Window Log	Log timestamp; count in window	Accurate	High memory (sorted set)
Sliding Window Counter	Interpolate prev+current window	Approximate, efficient	Slight inaccuracy

Interview Q&A - API Gateway & Service Discovery

Q1. What is the primary purpose of an API Gateway?

An API Gateway provides a single entry point for all client requests, offloading cross-cutting concerns - authentication (JWT/OAuth2 validation), rate limiting, SSL termination, request routing, and load balancing - from individual microservices, reducing duplication and simplifying the service layer.

Q2. How does JWT validation work at the API Gateway?

The gateway intercepts each request, extracts the Authorization header, decodes the JWT, verifies the signature using the public key (RS256) or shared secret (HS256), checks exp/nbf claims, and either forwards with enriched headers (user-id, roles) or rejects with HTTP 401/403 without touching downstream services.

Q3. Explain the token bucket rate-limiting algorithm.

A bucket holds up to N tokens. Tokens are added at rate r tokens/second. Each request consumes one token. If the bucket is empty the request is rejected (HTTP 429). The burst capacity equals the bucket size. This allows short bursts while enforcing a long-term average rate.

Q4. What is SSL termination at the gateway?

SSL/TLS is decrypted at the gateway, so downstream microservices communicate over plain HTTP within the private network. This reduces CPU load on services, centralises certificate management, and simplifies mTLS policies.

Q5. Compare Kong and AWS API Gateway.

Kong is open-source, plugin-based (Lua/Go), runs on-prem or in K8s, and offers fine-grained control. AWS API Gateway is fully managed, natively integrates with Lambda/IAM/Cognito, and requires no infrastructure management but is vendor-locked to AWS with per-call pricing.

Q6. What is service discovery and why is it needed?

In a microservice environment, service instances start and stop dynamically with changing IP/port. Service discovery provides a registry where services register on start and deregister on stop, enabling consumers to find healthy instances at runtime without static configuration.

Q7. Explain client-side service discovery.

The service client queries a registry (Eureka, Consul) directly to retrieve a list of healthy instances, then applies its own load-balancing algorithm (round-robin) to pick an instance and call it directly.

Q8. Explain server-side service discovery.

The client sends requests to a load balancer (AWS ALB, Kubernetes Service DNS). The LB queries the registry internally and routes to a healthy instance. The client is unaware of the registry; it only knows the LB address.

Q9. What is Eureka's self-preservation mode?

If Eureka receives fewer heartbeats than expected (e.g., network partition), it enters self-preservation and stops evicting instances whose leases have expired. This preserves availability - the registry keeps potentially stale data - rather than aggressively removing healthy instances that merely cannot reach the server.

Q10. How does Consul differ from Eureka?

Consul is CP (Raft consensus, strong consistency), supports active health checks (HTTP/TCP/script), has a built-in KV store for config, and supports multi-datacenter out of the box. Eureka is AP (availability first), uses heartbeat TTL, and is Java-only. Consul also provides a DNS interface alongside its HTTP API.

Q11. Describe the three circuit breaker states.

CLOSED: normal operation, requests pass through, failure count tracked. OPEN: failure rate exceeded threshold; all requests fail fast with fallback, no upstream calls made. HALF-OPEN: after a wait duration, a small number of probe requests are allowed to test recovery; success transitions to CLOSED, failure returns to OPEN.

Q12. What triggers a circuit breaker to OPEN?

When the failure rate (exceptions or timeouts as a percentage of calls) in a sliding window exceeds a configured threshold (e.g., 50% over the last 10 calls), Resilience4j transitions to OPEN. A slow call rate threshold can also trigger OPEN even if no exceptions occur.

Q13. What happens during HALF-OPEN state?

A configured number of probe calls (`permittedNumberOfCallsInHalfOpenState`) are allowed through. Resilience4j evaluates their success/failure rate. If recovery is confirmed (below threshold), the breaker transitions to CLOSED; if failures continue, it returns to OPEN and resets the wait timer.

Q14. How do you implement a fallback with Resilience4j?

Annotate the method with `@CircuitBreaker(name="...", fallbackMethod="fallback")`. The fallback method must have the same parameters plus a Throwable. It can return a cached value, default response, or error DTO. For reactive flows, use `ReactiveCircuitBreaker` from Resilience4j Reactor.

Q15. What is the difference between circuit breaker and retry?

Retry re-attempts a failed call up to N times with optional backoff - useful for transient errors. Circuit breaker stops all calls to a failing service immediately to prevent cascading failures - useful when the service is persistently unhealthy. They are complementary: retry handles occasional blips; circuit breaker handles sustained failure.

Q16. How does request routing work in Spring Cloud Gateway?

Routes are defined with predicates (Path, Host, Method, Header) and filters (StripPrefix, AddRequestHeader, RewritePath, RequestRateLimiter). The gateway evaluates predicates in order and forwards matching requests to the uri (lb:// for load-balanced Eureka lookup or direct URL).

Q17. What is the difference between 302 and 301 redirects in an API context?

301 Moved Permanently is cached by browsers/CDNs indefinitely. 302 Found is temporary - every request goes through the gateway, enabling analytics tracking, A/B routing, and controlled redirects. Most URL shorteners use 302 for analytics.

Q18. How does rate limiting work with Redis in a distributed gateway cluster?

Each gateway node uses a shared Redis instance. The token bucket or sliding window counter is stored as Redis keys (INCR+EXPIRE or sorted sets). All nodes atomically read/write the same counter via Redis, ensuring rate limits are enforced globally across the cluster rather than per-node.

Q19. What is request transformation in an API gateway?

Transformation includes: adding/removing headers (inject user-id from JWT, strip internal headers), rewriting paths (e.g., `/v1/orders` to `/api/orders`), protocol translation (REST to gRPC), payload transformation (XML to JSON), and response mapping (normalise error formats). Kong uses Lua plugins; Spring Cloud Gateway uses filters.

Q20. Explain health checks in Consul vs Eureka.

Consul performs active health checks at configured intervals - HTTP GET to `/health`, TCP connection, or script execution. If checks fail, the instance is marked critical and excluded from DNS/API responses. Eureka relies on passive heartbeat TTL (default 90s); if the service stops sending heartbeats, it is evicted after the TTL unless self-preservation kicks in.

Q21. What is load balancing at the gateway level?

The gateway distributes requests across instances using algorithms: Round-Robin (equal distribution), Weighted Round-Robin (proportion by weight), Least Connections (route to instance with fewest active requests), IP Hash (sticky sessions by client IP), or Random. Spring Cloud Gateway uses Spring Cloud LoadBalancer integrating with Eureka for dynamic instance lists.

Q22. How do you secure an API gateway?

Apply: mTLS between gateway and services; WAF rules (SQL injection, XSS patterns); IP allowlisting; JWT/API-key validation; rate limiting per key; DDoS protection at the edge (CloudFront, Cloudflare); OWASP API Top 10 mitigations; audit logging of all requests with correlation IDs.

Q23. What is a service mesh and how does it differ from an API gateway?

A service mesh (Istio, Linkerd) handles east-west (service-to-service) traffic with a sidecar proxy per pod, providing mTLS, observability, and circuit breaking at the network layer without code changes. An API gateway handles north-south (external-to-internal) traffic. They are complementary.

Q24. How does Kong's plugin architecture work?

Kong is built on Nginx+OpenResty. Plugins are Lua (or Go/Python via PDK) scripts that hook into Kong's request lifecycle phases: `access` (auth, rate limit), `header_filter` (transform headers), `body_filter` (transform body), `log` (audit). Plugins are enabled per route/service/globally via the Admin API or declarative YAML (deck).

Q25. What is canary deployment via API gateway?

The gateway routes a small percentage of traffic (e.g., 5%) to a new service version based on weight or headers, while 95% goes to the stable version. This enables gradual rollout with real traffic, monitoring error rates and latency before full promotion. Implemented via weighted routes in Spring Cloud Gateway or Kong traffic-split plugin.

Q26. How do you handle timeout configuration in a gateway?

Set connect-timeout (time to establish TCP connection, e.g., 2s), response-timeout (time to receive first byte, e.g., 10s), and read-timeout (idle time between bytes). Timeouts should be shorter than client timeouts to allow proper error propagation. Combine with circuit breaker to avoid thread exhaustion on slow upstreams.

Q27. What is the Bulkhead pattern and how does it relate to circuit breaking?

Bulkhead isolates concurrent calls to a service into a fixed-size thread pool or semaphore. If all permits are taken, new calls are rejected immediately, preventing one slow service from consuming all threads. Circuit breaker detects failure rate over time. Together: bulkhead limits concurrent load, circuit breaker handles

sustained failures.

Q28. How do you propagate correlation IDs through a gateway?

The gateway generates (or extracts from incoming request) a unique correlation ID (UUID), injects it as X-Correlation-ID header into the forwarded request, and logs it. Downstream services extract the header into MDC (MDC.put("correlationId", id)) so all log lines include it, enabling end-to-end request tracing across services.

Q29. What is the difference between stateless and stateful rate limiting?

Stateless rate limiting (per-node counters, no shared state) is fast but inaccurate in a cluster. Stateful rate limiting (shared Redis counter) is accurate globally but adds Redis latency (~1ms). For most use cases, shared Redis is preferred. For ultra-low latency, local approximate counters with periodic sync can be used.

Q30. How do you implement blue-green deployment with an API gateway?

Two identical environments (blue=current, green=new) run simultaneously. The gateway is configured to route 100% traffic to blue. After validating green, the gateway switches routing to green (100%). Rollback is instant - switch back to blue. No downtime. Contrast with canary which splits traffic gradually.

Q31. What is CORS and how does the gateway handle it?

Cross-Origin Resource Sharing is a browser security policy. The gateway adds Access-Control-Allow-Origin, Access-Control-Allow-Methods, Access-Control-Allow-Headers headers to responses. Preflight OPTIONS requests are handled at the gateway level, avoiding the need for each microservice to implement CORS logic.

Q32. Explain the difference between gateway and reverse proxy.

A reverse proxy (Nginx, HAProxy) forwards requests to upstream servers with minimal processing - mainly routing and load balancing. An API gateway is a specialised reverse proxy that adds API-specific concerns: authentication, rate limiting, request transformation, API key management, developer portal integration, and analytics.

Q33. How does AWS API Gateway integrate with Lambda?

API Gateway triggers a Lambda function synchronously (REST API or HTTP API). The request is mapped to a Lambda event JSON. The Lambda processes the business logic and returns a response which API Gateway maps back to HTTP. This serverless model eliminates server management but has cold start latency (mitigated with provisioned concurrency).

Q34. What is the Sidecar pattern in the context of service discovery?

A sidecar proxy (Envoy in Istio) runs alongside each service instance and handles all network communication. It registers/deregisters with the control plane, performs health checks, load balancing, and circuit breaking transparently. The application code calls localhost:PORT and the sidecar handles routing.

Q35. How do you handle WebSocket connections at the API gateway?

For WebSockets, the gateway must support protocol upgrade (HTTP to WS). Spring Cloud Gateway supports WebSocket proxying. For long-polling, ensure connection timeouts are set longer than the poll interval. AWS API Gateway v2 (HTTP API) natively supports WebSocket routes.

Q36. What is API versioning strategy at the gateway level?

Common strategies: URL path versioning (/v1/orders, /v2/orders) - most visible, easy to route; header versioning (Accept: application/vnd.api.v2+json) - cleaner URLs, harder to test; query param (?version=2) -

simple but pollutes URLs. Gateway routes to appropriate service version based on path prefix, header, or query param predicates.

Q37. How does service deregistration work and what are the risks?

On graceful shutdown (SIGTERM), the service calls the registry to deregister. In Eureka, the TTL (leaseExpirationDurationInSeconds) also causes eviction if heartbeats stop. Risk: if a service crashes without deregistering, the registry keeps a stale entry until TTL expires. During this window, client-side LB may route to a dead instance - mitigated by health checks or retry logic.

Q38. What is the difference between synchronous and asynchronous API gateway patterns?

Synchronous: gateway forwards request and waits for response (standard HTTP). Asynchronous: gateway accepts the request, publishes to a message queue (Kafka/SQS), returns 202 Accepted with a job ID. Client polls or uses WebSocket for the result. Async is appropriate for long-running operations to avoid timeout issues.

Q39. How do you implement API key management at the gateway?

API keys are issued per consumer (application or developer). The gateway validates the key against a store (Redis, database) on each request, enforces per-key rate limits and quotas, and associates the key with a consumer identity for analytics. Kong Key Auth plugin or Spring Security can implement this pattern.

Q40. What is the Strangler Fig pattern and how does the gateway enable it?

The Strangler Fig pattern incrementally migrates a monolith to microservices. The gateway sits in front of both the monolith and new microservices. Initially all traffic goes to the monolith. As each domain is extracted to a microservice, the gateway routes that subset of requests to the new service. Eventually the monolith is retired.

Q41. How do you monitor an API gateway in production?

Key metrics: request rate (RPS), error rate (4xx/5xx ratio), latency percentiles (p50/p95/p99), rate-limit rejections, circuit breaker state changes, active connections. Use Prometheus+Grafana (Kong Prometheus plugin, Spring Boot Actuator + Micrometer), distributed tracing (propagate traceId), and centralised logging with correlation IDs.

Q42. Explain sliding window rate limiting with Redis sorted sets.

Store each request timestamp as a member of a sorted set (key=user:ratelimit, score=timestamp). To check: remove members older than window (ZREMRANGEBYSCORE), count remaining (ZCARD). If count < limit, add current timestamp (ZADD) and allow; else reject. This gives exact count in sliding window at cost of $O(\log N)$ per request.

Q43. What is mutual TLS (mTLS) and when would you use it?

mTLS requires both client and server to present certificates. At the gateway: the gateway presents a server cert (standard TLS) and also verifies a client cert. Used for: service-to-service authentication in a zero-trust network, B2B API clients requiring strong identity verification, and securing internal gateway-to-service communication.

Q44. How does Kong's declarative configuration (deck) work?

deck (Deck) is a CLI tool to manage Kong configuration as YAML files stored in version control. You define services, routes, plugins, and consumers in YAML. deck diff shows changes; deck sync applies them to Kong's Admin API idempotently. This enables GitOps for API configuration - changes are reviewed via PR

and applied by CI/CD.

Q45. What are the trade-offs of having too many plugins/filters in the gateway?

Each plugin/filter adds processing latency. A chain of 10 plugins (auth, rate limit, transform, log, cache, etc.) can add 5-15ms per request. Mitigations: profile plugin overhead, disable unused plugins, use asynchronous logging, cache JWT validation results (short TTL), and horizontally scale the gateway.

Q46. How do you handle authentication for internal microservice calls?

Options: (1) JWT propagation - gateway validates and forwards the JWT; downstream services verify the same token. (2) Service account tokens - each service has its own JWT/certificate. (3) mTLS - mutual cert verification at the network layer. (4) Service mesh - Istio handles mTLS transparently.

Q47. What is the difference between rate limiting and throttling?

Rate limiting enforces a maximum request rate per time window (e.g., 100 req/min) - excess requests are rejected (429). Throttling slows down request processing (adds delay) rather than rejecting, protecting the server from overload while maintaining availability. Rate limiting provides hard boundaries; throttling is less aggressive.

Q48. How does Spring Cloud LoadBalancer replace Ribbon?

Spring Cloud LoadBalancer is the replacement for the deprecated Netflix Ribbon. It integrates with Spring Cloud DiscoveryClient (Eureka, Consul) to get instance lists, and supports RoundRobinLoadBalancer (default) and RandomLoadBalancer. Configure via `@LoadBalancerClient` annotation or `ReactiveLoadBalancer` for WebFlux.

Q49. What is the role of the Actuator /health endpoint in service discovery?

Spring Boot Actuator's `/actuator/health` endpoint is used by Eureka/Consul for health checks. If the endpoint returns DOWN, the registry marks the instance as unhealthy and stops routing traffic to it. Configure health indicators for database, Redis, downstream services. Kubernetes liveness/readiness probes also use this endpoint.

Q50. How do you handle circuit breaker state in a microservice cluster?

Circuit breaker state is local per instance - each JVM has its own Resilience4j state machine. If one instance opens its circuit, others may still be CLOSED. This is by design - distributed circuit state adds consensus overhead. For visibility, publish circuit state events to Prometheus metrics (`resilience4j_circuitbreaker_state`) and alert on widespread OPEN states.

Q51. What are the trade-offs of client-side vs server-side service discovery?

Client-side: more flexible load balancing, fewer network hops, but increases client complexity and requires each client to know the registry. Server-side: cleaner clients, centralized LB logic, but introduces the LB as a potential bottleneck and single point of failure (mitigated with clustering).

Q52. How do you implement graceful degradation when the registry is unavailable?

Use client-side caching: Eureka client caches the last-known instance list locally. If the registry is unreachable, it continues using the cache (with staleness risk). Consul has similar local agent caching. Configure retry with exponential backoff for registry queries. Set a reasonable cache TTL (e.g., 30s) to balance freshness vs resilience.
